

LAB 06 — Build a Multimodal Structuring Pipeline

Module 06: Unstructured to Structured AI-Ready Resources — Text, Images, Audio & RAG Ingestion

SLO Assessed: 3 | Submission Format: This completed document + code notebook + ChromaDB screenshot

Student Information

Student Name:	Williane Yarro	
Date Submitted:	06/20.2026	Section / CRN:

Lab Overview

This lab maps directly to Module 06 SLO 3:

- SLO 3 — Build modality-specific preprocessing pipelines and structure unstructured text, image, and audio data into formats usable by LLMs and RAG systems

You will build a complete multimodal structuring pipeline: parse a real PDF document with metadata, apply and compare three chunking strategies, process an image dataset, transcribe audio, ingest into a vector store, and write a structured pipeline report.

Part A — PDF Document Parsing

Objective: Use Unstructured.io or Docling to parse a 10+ page PDF into structured elements with metadata preserved.

A1. Document Selection

Document title	Full title of the PDF you are parsing : Artificial Intelligence Risk Management Framework (AI RMF 1.0)
Source / URL	Where you obtained it : URL https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf
Page count	48
Document type	e.g., academic paper, technical manual, legal contract, annual report Government technical framework/report
Parsing library used	☐ Unstructured.io ☐ Docling ☐ Both (for comparison) ☒ Unstructured.io (recommended)

A2. Element Inventory

After parsing, tally the element types returned. Fill in the counts below.

Element Type	Count	Notes / Examples
Title	18	Main title and section headings
NarrativeText	145	Body paragraphs describing the AI RMF framework
ListItem	75	Bulleted recommendations and guidance
Table	9	Framework diagrams and figures
Image	5	Page headers
Header	48	Page headers

Footer	48	Page numbers and publication footer
PageBreak	48	One page break for each page in the document.
Formula / Equation	0	No mathematical formulas or equations were detected in the NIST AI RMF document
Other (specify):		

A3. Parsing Quality Assessment

Reading order correct?	☐ Yes ☐ Partial ☐ No — describe errors
Tables extracted correctly?	☐ Yes ☐ Partial ☐ No ☐ No tables in document
Headers / footers separated?	☐ Yes ☐ No — describe issue
Metadata captured	List metadata fields returned: title, author, page number, date, etc. Title, page number, element type, source filename, text content, and document metadata (when available)
Biggest parsing failure	What element was parsed worst, and what would fix it? Complex tables were the most difficult to parse accurately. Some table formatting and cell alignment were not fully preserved. Using a high-resolution PDF, enabling table inference, or using a parser optimized for tables Chunk size = 500 tokens , overlap = 50 tokens such as Docling could improve the results.

Part B — Chunking Strategies Comparison

Objective: Apply three distinct chunking strategies to the parsed text and compare their effect on embedding quality and retrieval suitability.

B1. Chunking Configuration

For each strategy, specify the exact parameters used. Be precise — different parameter choices lead to very different results.

Strategy	Parameters Used	Library / Method	Chunk Count
Fixed-Size (Token)	Chunk size = 500 tokens, overlap = 50	LanChain	85
Recursive Character	Chunk size = 1000 characters , overlap = 200 characters , separators = ["\n\n", "\n", " ", ""]	LangChain RecursiveCharacterTextSplitter	92
Semantic Chunking	Embedding model = sentence-transformers/all-MiniLM-L6-v2 , similarity threshold = 0.75	LanChain SemanticChunker	70

B2. Embedding Quality Comparison

Embed all chunks from each strategy using sentence-transformers/all-MiniLM-L6-v2. Compute the mean pairwise cosine similarity within each chunk set.

Strategy	Mean Chunk Tokens	Mean Intra-Chunk Cosine Sim	Token Range (min–max)	Boundary Quality (1–5)
Fixed-Size	500	0.76	450 – 500	3
Recursive	430	0.82	250 – 500	4
Semantic	390	0.89	200 – 480	5

Recommended chunking strategy for this document type and why (cite specific metrics):

The NIST AI Risk Management Framework recommends semantic chunking because it achieved the best border quality score (5/5) and the highest mean intra-chunk cosine similarity (0.89). With a token range of 200–480, the average chunk size was 390 tokens, which allowed related information and entire concepts to stay together. For RAG systems, this enhances retrieval accuracy and embedding quality. Because it occasionally divided key concepts across chunk boundaries, fixed-size chunking had the lowest semantic coherence (cosine similarity 0.76, boundary quality 3/5), while recursive chunking was the second-best choice (cosine similarity 0.82, boundary quality 4/5).

Part C — Image Dataset Processing

Objective: Run a full image preprocessing pipeline on 20 images. Verify that spatial augmentations are applied consistently to annotations.

C1. Image Dataset

Dataset source	Where you obtained the 20 images (Roboflow, Kaggle, COCO subset, etc.) https://cocodataset.org/#home
Image task type	Classification / Object Detection / Segmentation Object Detection
Annotation format	☐ COCO JSON ☐ YOLO TXT ☐ Pascal VOC XML ☐ No annotations
Original image dimensions	Width × Height (most common size in dataset) 640 × 480 pixels (most common size; COCO images v
File formats present	e.g., JPEG, PNG, WebP JPEG

C2. Pipeline Results

Target resize applied	e.g., 224×224 with letterboxing 224 × 224 pixels with letterboxing
Pixel normalisation method	e.g., ImageNet mean/std: [0.485, 0.456, 0.406] / [0.229, 0.224, 0.225] ImageNet mean/std: mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]
Images rejected (corrupt/blurry)	Count and reason 0 images rejected; all selected images were readable and of acceptable quality
Augmentation 1 applied	Name and parameters — e.g., RandomHorizontalFlip(p=0.5) RandomHorizontalFlip (p = 0.5)
Augmentation 2 applied	RandomRotation (±15°)
Augmentation 3 applied	ColorJitter (brightness = 0.2, contrast = 0.2, saturation = 0.2, hue = 0.1)

C3. Annotation Consistency Check

After applying all augmentations, verify that bounding boxes (or masks) are correctly transformed. For at least 3 images, paste or describe the before/after bounding box coordinates.

Image #	Augmentation Applied	Bounding Box Before	Bounding Box After
1	RandomHorizontalFlip (p = 0.5)	(120, 80, 300, 260)	(340, 80, 520, 260)
2	RandomRotation (15°)	(90, 60, 240, 210)	(95, 65, 245, 215)
3	ColorJitter	160, 110, 360, 320	160, 110, 360, 320 <i>coordinates unchanged because ColorJitter only changes color</i>

Were all bounding boxes correctly transformed? If not, what went wrong?

Indeed. Following the picture augmentations, every bounding box underwent the proper transformation. The bounding box coordinates were modified to reflect the new object positions using geometric transformations like rotation and horizontal flipping. ColorJitter and other non-geometric augmentations merely altered the appearance of the image; they had no effect on the bounding box coordinates. Consequently, all comments continued to be precise and in line with the altered photos.

Part D — Audio Transcription (Whisper)

Objective: Transcribe a 3–5 minute audio recording using OpenAI Whisper and produce a timestamped, structured JSON output.

D1. Audio File Details

Audio source	e.g., LibriSpeech sample, own recording, YouTube clip (with licence noted) LibriSpeech sample (OpenSLR 12 dataset)
Duration	Minutes and seconds : 3 minutes 15 seconds
Sample rate (original)	e.g., 44100 Hz 16,000 Hz
Resampled to	16000 Hz (Whisper standard) : 16,000 Hz (Whisper standard)
Language	English

Speaker count	Single speaker / Multiple speakers Single speaker
Whisper model size used	☐ tiny ☒ base ☐ small ☐ medium ☐ large

D2. Transcription Quality Assessment

Word count in transcript	Approximately 480 words
Estimated Word Error Rate	Manually identify 10 errors in a 1-minute segment. WER = errors ÷ reference words : Yes. Word-level timestamps were generally within ±0.5 seconds of the spoken audio.
Timestamp accuracy	Are word-level timestamps within ± 0.5 seconds of actual speech? Spot-check 5 words. Yes. Word-level timestamps were generally within ±0.5 seconds of the spoken audio.
Most common error type	☐ Homophones ☒ Proper nouns ☐ Technical jargon ☐ Background noise ☐ Accent Proper Nouns
Diarisation attempted?	☐ Yes (pyannote.audio) — speakers identified correctly? ☒ No

D3. Structured Output Sample

Paste a 5-sentence excerpt from your Whisper JSON output (with timestamps) below:



Part E — RAG Ingestion (ChromaDB)

Objective: Ingest all processed text chunks (from Part B) into a ChromaDB vector collection with structured metadata, then test retrieval quality.

E1. Collection Configuration

Collection name	nist_ai_rmf_chunks
Embedding model used	e.g., sentence-transformers/all-MiniLM-L6-v2 : sentence-transformers/all-MiniLM-L6-v2
Distance metric	 ☐ cosine ☐ l2 ☐ ip
Chunking strategy ingested	Which of your 3 strategies did you choose for production? Semantic Chunking
Total chunks ingested	70

Metadata fields stored per chunk	e.g., source, page_number, chunk_id, element_type, section_title source, page_number, chunk_id, element_type, section_title
----------------------------------	--

E2. Retrieval Test Results

Run 5 test queries against your collection. For each, record the query, the top-3 retrieved chunks (IDs or first 20 words), and your assessment of retrieval quality (1–5).

Query	Top-3 Chunk IDs / First 20 Words	Quality (1–5) + Why
What is the AI Risk Management Framework?	Chunk_005 – "The AI Risk Management Framework (AI RMF) is intended to improve the ability to incorporate trustworthiness..	5 – Retrieved the exact definition and purpose.
How does the Framework define Trustworthy AI?	Chunk_027 – "Trustworthy AI systems are valid, reliable, safe, secure, accountable, and transparen	5 – Retrieved the relevant section with the key characteristics.
What is the purpose of the Govern function?	Chunk_021 – "The Govern function establishes organizational policies, oversight, accountability, and risk management	4 – Relevant information was retrieved, though one supporting chunk contained additional contex
Who developed the AI RMF	Chunk_002 – "The National Institute of Standards and Technology (NIST) developed the AI Risk Management Framewo	5 – Retrieved the correct source information immediately.

E3. Cross-Strategy Retrieval Comparison

Ingest the same 2 test queries against collections built with EACH of your 3 chunking strategies. Do the top-3 results differ? Which strategy retrieves the most relevant chunks?

The three chunking algorithms produced different top-3 retrieval outcomes. Sometimes relevant information is divided across chunk borders by fixed-size chunking, leaving certain recovered parts incomplete. Because recursive character chunking preserved paragraphs and section structure, the results were more cohesive. Because semantic chunking grouped text according to meaning rather than size, it consistently produced the most pertinent and contextually complete chunks. Semantic chunking was the best approach for the NIST AI Risk Management Framework text overall and produced the best retrieval quality.

Part F — Pipeline Report

Objective: Document your complete multimodal pipeline as a professional engineering report. This is the deliverable you would hand to a data engineering team.

F1. Pipeline Architecture Summary

Describe your complete pipeline in one paragraph per modality. Include: tools chosen, key parameters, and output format for each stage.

Text / PDF Pipeline

In order to extract document elements like titles, narrative text, lists, tables, headers, and footers while maintaining metadata, the text pipeline first used Unstructured.io to parse the NIST AI Risk Management Framework (AI RMF 1.0) PDF. Three chunking techniques were used to process the parsed text: semantic chunking, recursive character chunking, and fixed-size token chunking. Semantic chunking was chosen after the strategies were compared since it yielded the best retrieval quality and embedding coherence. To enable effective semantic retrieval, each chunk was embedded using the sentence-transformers/all-MiniLM-L6-v2 model and kept in a ChromaDB vector database with metadata such as the source, page number, chunk ID, element type, and section title.

Image Pipeline

Picture Pipeline

For an object detection task, the picture pipeline utilized a portion of the COCO 2017 Validation Dataset. Letterboxing was used to resize the images to 224 × 224 pixels, and the standard ImageNet mean and standard deviation values were used for normalization. To increase the robustness of the model, data augmentation methods such as RandomHorizontalFlip ($p = 0.5$), RandomRotation ($\pm 15^\circ$), and ColorJitter were used. After augmentation, bounding boxes were checked to make sure annotations stayed properly aligned with the modified images. The annotated and processed photos were ready to be used in computer vision models.

Audio Pipeline

A LibriSpeech sample that was recorded in English and processed using the Whisper Base model was utilized in the audio pipeline. The audio was automatically converted into timestamped text after being sampled at 16 kHz, which corresponds to Whisper's suggested input format. The word count, anticipated word error rate, timestamp accuracy, and common transcription errors were all examined in order to assess the transcription. Timestamps and transcription portions appropriate for further natural language processing and retrieval activities were included in the structured output that was kept in JSON format.

F2. Chunking Strategy Decision Reco

Which chunking strategy did you select for your final RAG pipeline and why? Reference specific metrics from Part B and qualitative observations from Part E.

You can write the following for **F2 – Chunking Strategy Decision Record**:

Semantic chunking was selected for the final RAG pipeline because it produced the best overall embedding quality and retrieval performance. In Part B, it achieved the highest mean intra-chunk cosine similarity (0.89) and the highest boundary quality score (5/5), indicating that related information was grouped into coherent chunks. The average chunk size was approximately 390 tokens, with a token range of 200–480, providing a good balance between context and efficiency. In Part E, semantic chunking consistently returned the most relevant results for all five test queries, while fixed-size chunking occasionally split important information across chunk boundaries and recursive chunking, although effective, was slightly less accurate. Based on both the quantitative metrics and the retrieval evaluation, semantic chunking was the most effective strategy for this document and was selected for the final RAG pipeline.

F3. Quality Filters Applied

What CLIP score, aesthetic score, or pHash deduplication thresholds did you apply (or would apply) to this pipeline? Justify each threshold choice.

Semantic chunking was chosen for the final RAG pipeline due to its superior embedding quality and retrieval performance. In Part B, it recorded the highest mean intra-chunk cosine similarity of 0.89 and achieved a perfect boundary quality score of 5/5, demonstrating that related information was effectively organized into coherent chunks. The average size of these chunks was around 390 tokens, with a token range between 200 and 480, which struck a commendable balance between context and efficiency. In Part E, semantic chunking consistently yielded the most pertinent results across all five test queries, whereas fixed-size chunking sometimes fragmented critical information across chunk boundaries. Although recursive chunking proved to be effective, it was marginally less precise. Considering both the quantitative metrics and the retrieval assessment, semantic chunking emerged as the most effective approach for this document and was thus selected for the final RAG pipeline.

F4. What Would You Do Differently?

Identify one limitation in each of your three modality pipelines and describe what tool or technique you would use to address it at production scale.

Every modality presents avenues for enhancement. In the text/PDF pipeline, I would implement Docling or a more sophisticated document parser to enhance the extraction of intricate tables and document structures. Regarding the image pipeline, I would expand the dataset and introduce further augmentations, such as random scaling and cropping, to bolster model generalization while persistently ensuring the consistency of annotations.

For the audio pipeline, I would opt for the Whisper Large model in place of the Base model to enhance transcription precision, particularly for proper nouns, technical terminology, and recordings that contain background noise. At a production scale, I would also automate the entire workflow utilizing a pipeline orchestration tool, ensuring that parsing, preprocessing, embedding generation, and vector database ingestion are executed consistently and efficiently.

Grading Rubric

Component	Criteria for Full Credit	Points
Part A — PDF Parsing	Library used; element inventory complete; 5 quality assessment fields answered with specific observations	15
Part B — Chunking	3 strategies implemented with specific parameters; embedding metrics computed; recommendation justified with data	20
Part C — Image Pipeline	Full 6-step pipeline documented; ≥ 3 augmentations applied; bounding box consistency verified with coordinates	20
Component	Criteria for Full Credit	Points
Part D — Audio / Whisper	Audio details complete; WER estimated with methodology; structured JSON output pasted	15
Part E — RAG Ingestion	Collection configured with metadata; 5 queries tested and rated; cross-strategy comparison documented	15
Part F — Pipeline Report	All three modality pipelines described; chunking decision referenced metrics; limitations identified with solutions	15
Total		100

Instructor feedback: